

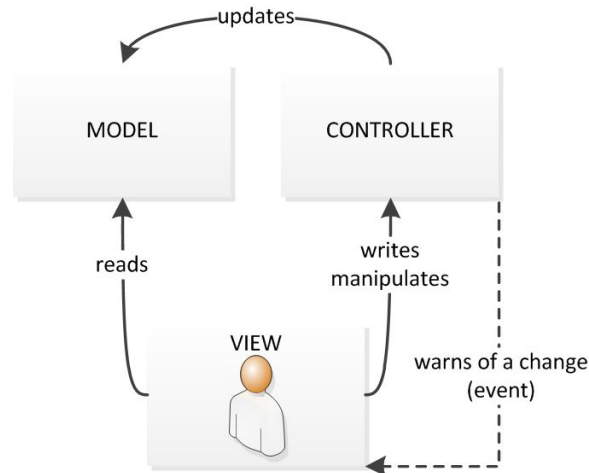
CSE470: Software Engineering

MVC Architecture



What is MVC Architecture

- MVC goes for Model-View-Controller Pattern
- Separates presentation and interaction from the data handling logic.
- It provides a extensible separation of concern (SOC) from presentation view to data processing logic.



Controller

- Handles all the user inputs through url
- The interaction to an application starts here by the user interactions – mouse click, key press etc
- Process http url requests (GET, POST, PUT, DELETE)
 - GET: for getting a data
 - POST: for posting / inserting a data
 - PUT: for updating a data
 - DELETE: for removing a data
- Communicates with both Model and View
- Contains all server side logic
- In the example – ProductController, UserController, AccountController etc.



Model

- It refers to the Data Related Logic
- Interaction with database (such as SELECT, INSERT, UPDATE, DELETE)
- It communicates with the controllers
- Can sometimes update or collaborate with the view (Depends on framework)
- In the example – Product, User, Transaction, Cart etc are model classes.

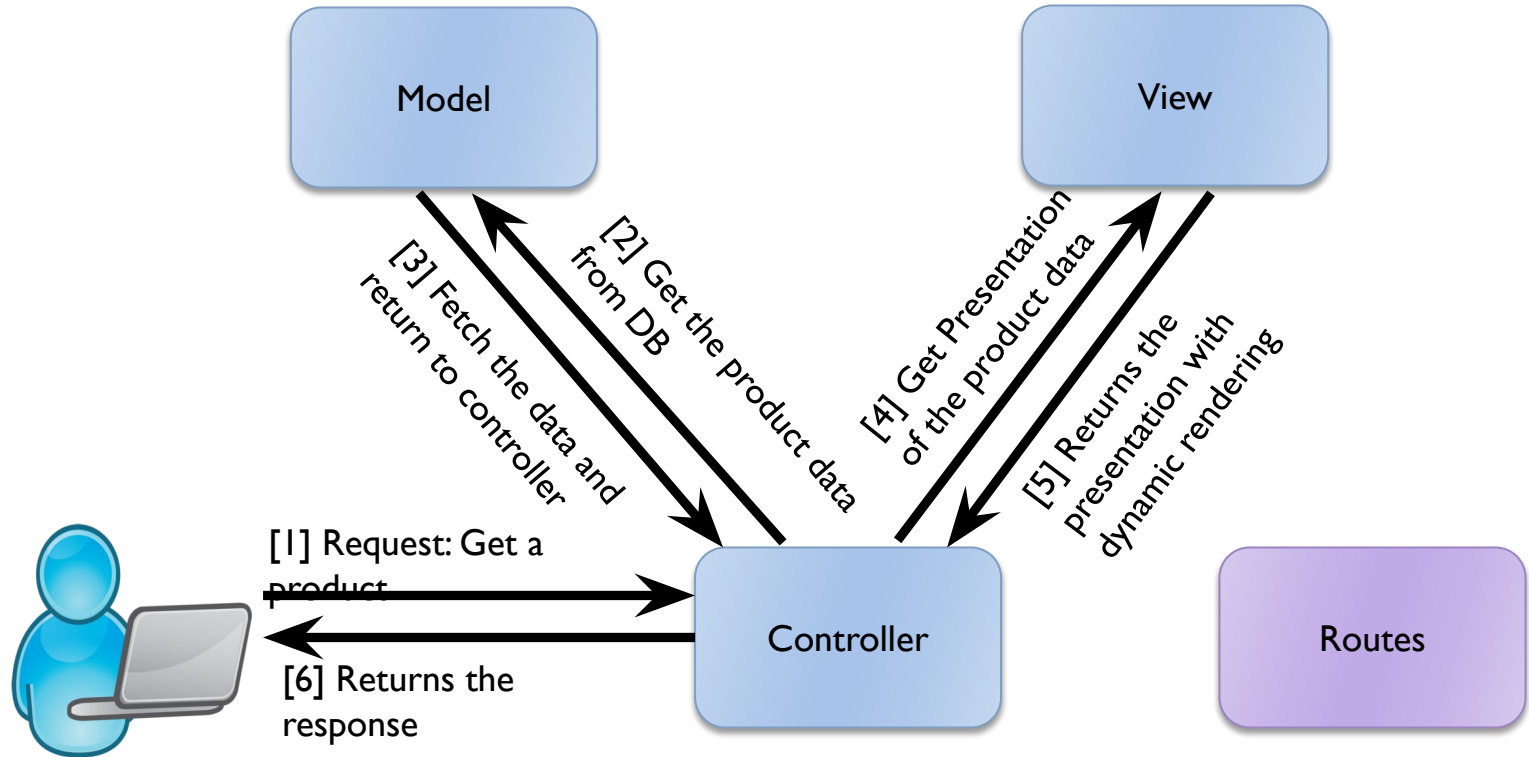


View

- What the end users see (UI)
- Usually Consists of html/css
- Communicates with the controller
- While coding are passed as dynamic values from the controller
- A controller can have multiple associated views.
- In the example – product.html, user.html to view a html file



MVC Steps



Routes

- Are urls to access a resource kept under App_Start folder
- General structure - `http://domainName/{controller}/{action}/{id}`
- `http://YourApp.com/Users/Profile/25`

```

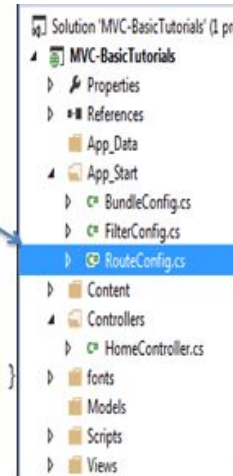
public class RouteConfig
{
    public static void RegisterRoutes(RouteCollection routes)
    {
        routes.IgnoreRoute("{resource}.axd/{*pathInfo}");

        routes.MapRoute(
            name: "Default",
            url: "{controller}/{action}/{id}",
            defaults: new { controller = "Home", action = "Index", id = UrlParameter.Optional }
        );
    }
}
  
```

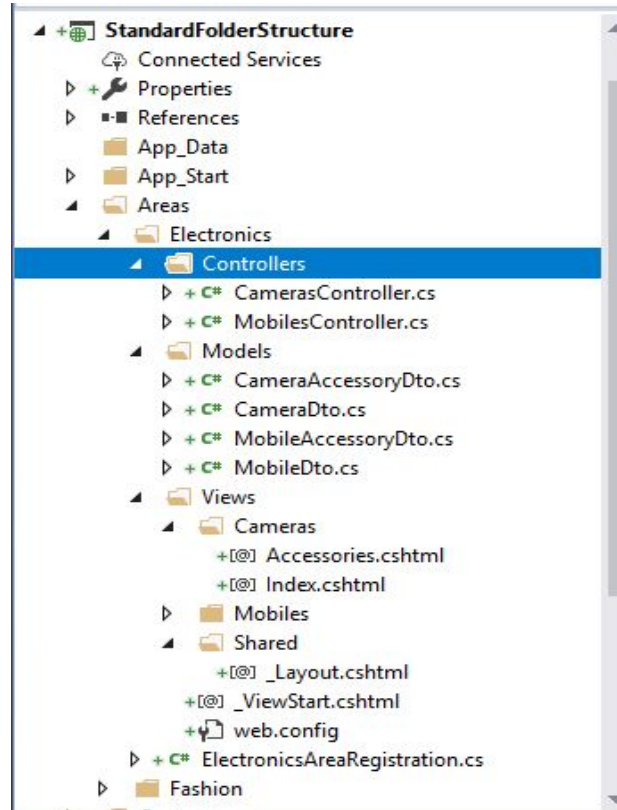
Annotations in the code:

- `routes.IgnoreRoute`: Route to ignore
- `name: "Default"`: Route name
- `url: "{controller}/{action}/{id}"`: URL Pattern
- `defaults: new { ... }`: Defaults for Route

RouteConfig.cs



MVC Folder Structure



MVC Code Structure

```
http://yourapp.com/users/profile/1

/routes
  users/profile/:id = Users.getProfile(id)

/controllers
  class Users{
    function getProfile(id){
      profile = this.UserModel.getProfile(id)

      renderView('users/profile', profile)
    }
  }

/models
  Class UserModel{
    function getProfile(id){
      data = this.db.get('SELECT * FROM users WHERE id = id')
      return data;
    }
  }

/views
  /users
    /profile
      <h1>{{profile.name}}</h1>
      <ul>
        <li>Email: {{profile.email}}</li>
        <li>Phone: {{profile.phone}}</li>
```

When to use MVC

Used when there are multiple ways to view and interact with data. Also used when the future requirements for interaction and presentation of data are unknown.

Note: Your final project MUST follow MVC Architecture



Advantages-Disadvantages

Advantages:

Allows the data to change independently of its representation and vice versa. Supports presentation of the same data in different ways with changes made in one representation shown in all of them. Other advantages are: parallel working, improved testability, code reusability

Disadvantages:

Can involve additional code and code complexity when the data model and interactions are simple.

JavaScript-based MVC Web App: [Click here](#)

